

230500N__a01

September 14, 2026

0.1 Department of Electronic and Telecommunication Engineering

0.1.1 University of Moratuwa

0.1.2 EN3160 Assignment 1 — Intensity Transformations and Neighborhood Filtering

Index No: 230500N

Name: Prishmika H.W.N

GitHub Repository: https://github.com/Nimindu04/EN3160_A01

0.2 Q 01

```
[2]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

def intensity_transform(im: np.ndarray, breakpoints: list) -> np.ndarray:
    bp = np.array(breakpoints, dtype=np.float32)
    x_in = np.arange(256)
    lut = np.interp(x_in, bp[:, 0], bp[:, 1]).astype(np.uint8)
    return cv.LUT(im, lut)

img = cv.imread('images/q1.jpeg', cv.IMREAD_GRAYSCALE) # Load image
breakpoints_list = [
    [[0, 0],[50, 50],[50, 100],[150, 255],[150, 150],[255, 255]],
    [[0, 0],[50, 20],[100, 50],[150, 100],[200, 150],[255, 200]],
    [[0, 0],[50, 80],[100, 130],[150, 180],[200, 220],[255, 255]]
]

fig, axes = plt.subplots(3, 3, figsize=(15, 12))
for row, breakpoints in enumerate(breakpoints_list):
    transformed_img = intensity_transform(img, breakpoints)
    bp = np.array(breakpoints)

    axes[row, 0].plot(bp[:, 0], bp[:, 1], 'r-o', linewidth=2)
    axes[row, 0].set_title(f'Transformation {row + 1}')
    axes[row, 0].set_xlabel('Input Intensity')
    axes[row, 0].set_ylabel('Output Intensity')
    axes[row, 0].set_xlim([0, 255])
    axes[row, 0].set_ylim([0, 255])
```

```

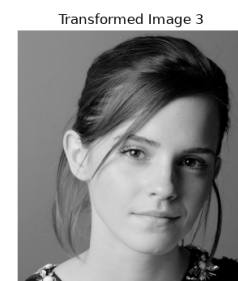
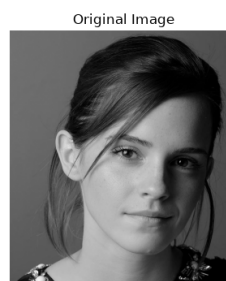
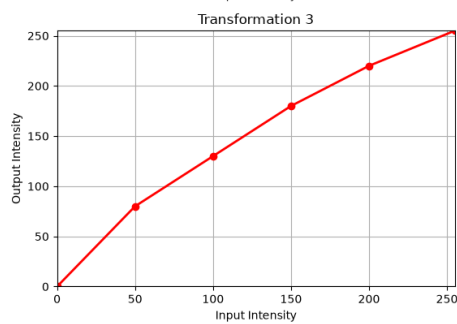
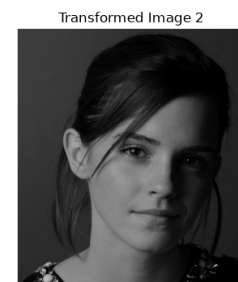
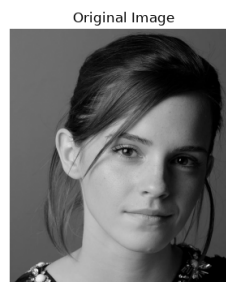
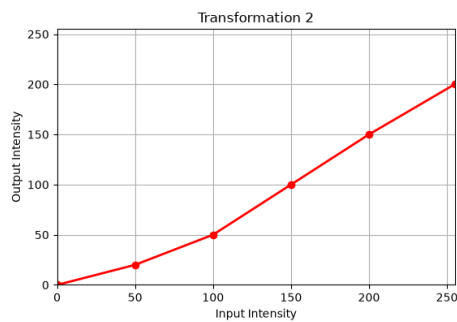
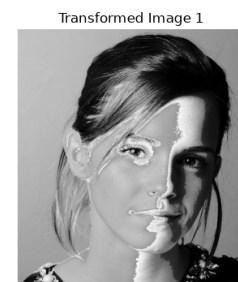
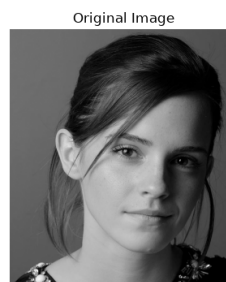
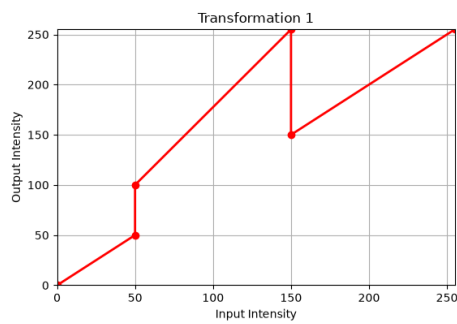
axes[row, 0].grid(True)

axes[row, 1].imshow(img, cmap='gray', vmin=0, vmax=255)
axes[row, 1].set_title('Original Image')
axes[row, 1].axis('off')

axes[row, 2].imshow(transformed_img, cmap='gray', vmin=0, vmax=255)
axes[row, 2].set_title(f'Transformed Image {row + 1}')
axes[row, 2].axis('off')

plt.tight_layout()
plt.show()

```



0.3 Q 02

```
[11]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

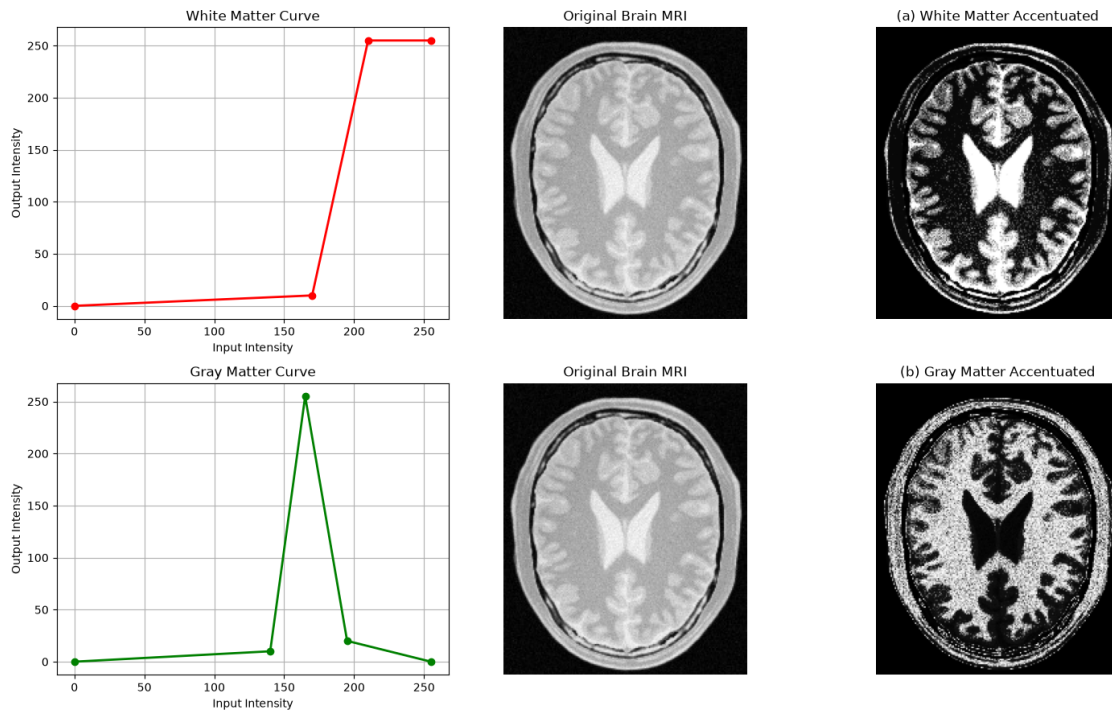
def intensity_transform(im: np.ndarray, breakpoints: list) -> np.ndarray:
    bp = np.array(breakpoints, dtype=np.float32)
    x = np.arange(256)
    lut = np.interp(x, bp[:, 0], bp[:, 1]).astype(np.uint8)
    return cv.LUT(im, lut)

img = cv.imread('images/q2.jpeg', cv.IMREAD_GRAYSCALE)
bp_white_matter = [[0, 0], [170, 10], [210, 255], [255, 255]]
bp_gray_matter = [[0, 0], [140, 10], [165, 255], [195, 20], [255, 0]]
img_wm = intensity_transform(img, bp_white_matter)
img_gm = intensity_transform(img, bp_gray_matter)

fig, axes = plt.subplots(2, 3, figsize=(15, 9)) #plot white matter
    ↪ transformation curve and images
bp_wm = np.array(bp_white_matter)
axes[0, 0].plot(bp_wm[:, 0], bp_wm[:, 1], 'r-o', linewidth=2)
axes[0, 0].set_title('White Matter Curve')
axes[0, 0].set_xlabel('Input Intensity')
axes[0, 0].set_ylabel('Output Intensity')
axes[0, 0].grid(True)
axes[0, 1].imshow(img, cmap='gray')
axes[0, 1].set_title('Original Brain MRI')
axes[0, 1].axis('off')
axes[0, 2].imshow(img_wm, cmap='gray')
axes[0, 2].set_title('(a) White Matter Accentuated')
axes[0, 2].axis('off')

bp_gm = np.array(bp_gray_matter) #plot gray matter transformation curve and
    ↪ images
axes[1, 0].plot(bp_gm[:, 0], bp_gm[:, 1], 'g-o', linewidth=2)
axes[1, 0].set_title('Gray Matter Curve')
axes[1, 0].set_xlabel('Input Intensity')
axes[1, 0].set_ylabel('Output Intensity')
axes[1, 0].grid(True)
axes[1, 1].imshow(img, cmap='gray')
axes[1, 1].set_title('Original Brain MRI')
axes[1, 1].axis('off')
axes[1, 2].imshow(img_gm, cmap='gray')
axes[1, 2].set_title('(b) Gray Matter Accentuated')
axes[1, 2].axis('off')
```

```
plt.tight_layout()
plt.show()
```



0.3.1 Histogram plots

```
[12]: import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

axes[0].hist(img.ravel(), 256, [0, 256], color='gray', alpha=0.8) ## 1.
    ↳Original Brain MRI Histogram
axes[0].set_title('Original Brain MRI Histogram')
axes[0].set_xlabel('Pixel Intensity')
axes[0].set_ylabel('Pixel Count')
axes[0].set_xlim([0, 255])
axes[0].grid(True, linestyle='--', alpha=0.5)

axes[1].hist(img_wm.ravel(), 256, [0, 256], color='crimson', alpha=0.8)
axes[1].set_title('White Matter Output Histogram')
axes[1].set_xlabel('Pixel Intensity')
axes[1].set_ylabel('Pixel Count')
axes[1].set_xlim([0, 255])
axes[1].grid(True, linestyle='--', alpha=0.5)
```

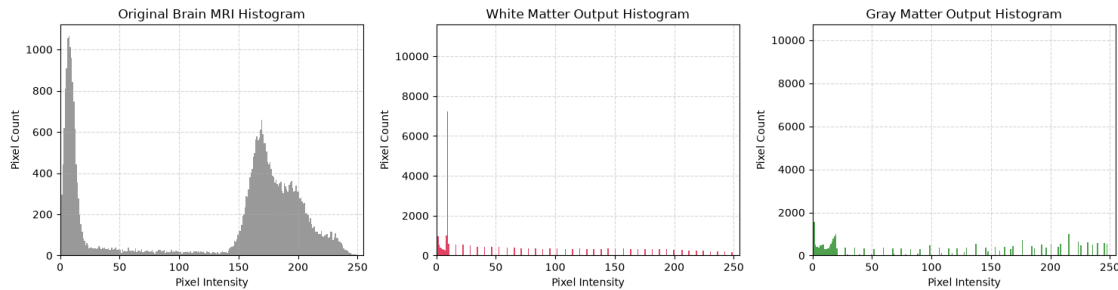


```

axes[2].hist(img_gm.ravel(), 256, [0, 256], color='forestgreen', alpha=0.8)
axes[2].set_title('Gray Matter Output Histogram')
axes[2].set_xlabel('Pixel Intensity')
axes[2].set_ylabel('Pixel Count')
axes[2].set_xlim([0, 255])
axes[2].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

```



Observation: Different intensity mappings can highlight specific brain tissues by selecting their intensity ranges.

0.4 Q 03

```

[8]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

img_bgr = cv.imread('images/q3.jpeg')
img_lab = cv.cvtColor(img_bgr, cv.COLOR_BGR2LAB) #image converted to LAB color space
l_plane, a_plane, b_plane = cv.split(img_lab)
gamma = 0.6
l_norm = l_plane / 255.0
l_corrected = np.clip(np.power(l_norm, gamma) * 255.0, 0, 255).astype(np.uint8)
img_lab_corrected = cv.merge([l_corrected, a_plane, b_plane])
img_rgb_corrected = cv.cvtColor(img_lab_corrected, cv.COLOR_LAB2RGB)
img_rgb_original = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)

fig, axes = plt.subplots(2, 2, figsize=(13, 8))

axes[0, 0].imshow(img_rgb_original)
axes[0, 0].set_title('Original Image (Fig. 3)')

```

```

axes[0, 0].axis('off')

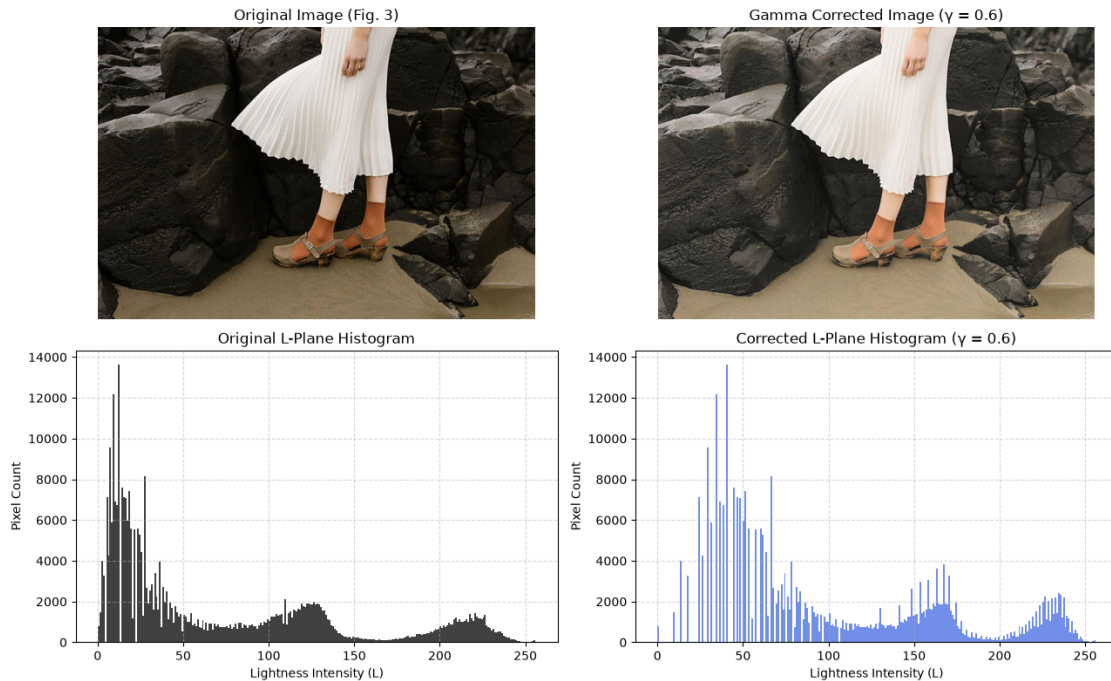
axes[0, 1].imshow(img_rgb_corrected)
axes[0, 1].set_title(f'Gamma Corrected Image ( = {gamma})')
axes[0, 1].axis('off')

axes[1, 0].hist(l_plane.ravel(), 256, [0, 256], color='black', alpha=0.75)
axes[1, 0].set_title('Original L-Plane Histogram')
axes[1, 0].set_xlabel('Lightness Intensity (L)')
axes[1, 0].set_ylabel('Pixel Count')
axes[1, 0].grid(True, linestyle='--', alpha=0.5)

axes[1, 1].hist(l_corrected.ravel(), 256, [0, 256], color='royalblue', alpha=0.75)
axes[1, 1].set_title(f'Corrected L-Plane Histogram ( = {gamma})')
axes[1, 1].set_xlabel('Lightness Intensity (L)')
axes[1, 1].set_ylabel('Pixel Count')
axes[1, 1].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

```



Answer: $\gamma = 0.6$ was used to gamma-correct the **L plane**.

Since $\gamma < 1$, it makes the dark and middle areas brighter. This helps to see more details in the dark rocks and shadow areas without making the white dress too bright.

0.5 Q 04

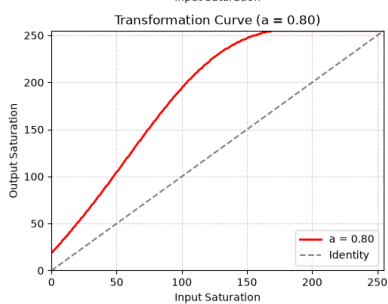
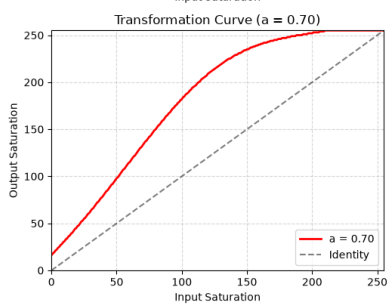
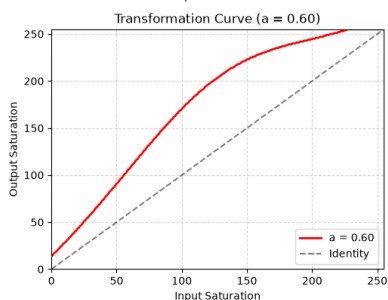
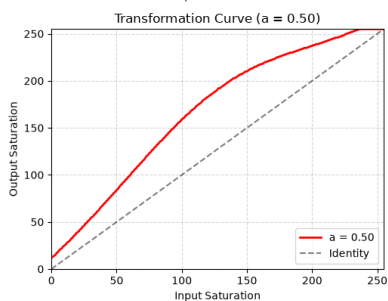
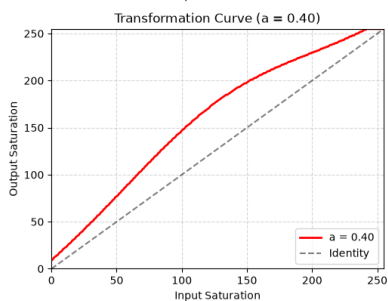
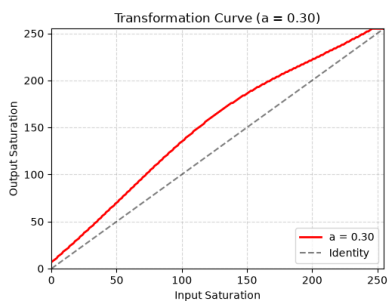
```
[ ]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img_bgr = cv.imread('images/q4.jpeg')
img_rgb_orig = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)
img_hsv = cv.cvtColor(img_bgr, cv.COLOR_BGR2HSV)
h_plane, s_plane, v_plane = cv.split(img_hsv)

def get_vibrance(a: float, sigma: float = 70.0) -> np.ndarray:
    x = np.arange(256, dtype=np.float32)
    gaussian_boost = a * 128.0 * np.exp(-((x - 128.0) ** 2) / (2.0 * (sigma ** 2)))
    return np.clip(x + gaussian_boost, 0, 255).astype(np.uint8)

a_values = [0.3, 0.4, 0.5, 0.6, 0.7, 0.8]
sigma_val = 70.0
x_axis = np.arange(256)
fig, axes = plt.subplots(len(a_values), 3, figsize=(15, 4 * len(a_values)))
for i, a in enumerate(a_values):
    lut = get_vibrance(a, sigma_val)
    s_enhanced = cv.LUT(s_plane, lut)
    hsv_enhanced = cv.merge([h_plane, s_enhanced, v_plane])
    img_rgb_enhanced = cv.cvtColor(hsv_enhanced, cv.COLOR_HSV2RGB)
    axes[i, 0].imshow(img_rgb_orig)
    axes[i, 0].set_title(f'Row {i+1}: Original Image')
    axes[i, 0].axis('off')
    axes[i, 1].imshow(img_rgb_enhanced)
    axes[i, 1].set_title(f'Enhanced Image (a = {a:.2f})')
    axes[i, 1].axis('off')
    axes[i, 2].plot(x_axis, lut, 'r-', linewidth=2, label=f'a = {a:.2f}')
    axes[i, 2].plot([0, 255], [0, 255], 'k--', alpha=0.5, label='Identity')
    axes[i, 2].set_title(f'Transformation Curve (a = {a:.2f})')
    axes[i, 2].set_xlabel('Input Saturation')
    axes[i, 2].set_ylabel('Output Saturation')
    axes[i, 2].set_xlim([0, 255])
    axes[i, 2].set_ylim([0, 255])
    axes[i, 2].grid(True, linestyle='--', alpha=0.5)
    axes[i, 2].legend(loc='lower right')

plt.tight_layout(); plt.show()
```



Best Value of (a) The best value of (a) was found to be **6**, as it produced the most visually pleasing output.

0.6 Q 05

```
[9]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

def histogram_equalization(im: np.ndarray) -> np.ndarray:

    hist, _ = np.histogram(im.flatten(), bins=256, range=[0, 256])
    cdf = hist.cumsum()
    total_pixels = im.size
    cdf_normalized = (cdf / total_pixels) * 255.0
    lut = np.round(cdf_normalized).astype(np.uint8)
    equalized_im = lut[im]
    return equalized_im

img = cv.imread('images/q5.tif', cv.IMREAD_GRAYSCALE)
img_equalized = histogram_equalization(img)

fig, axes = plt.subplots(2, 2, figsize=(13, 8))

axes[0, 0].imshow(img, cmap='gray', vmin=0, vmax=255)
axes[0, 0].set_title('Original Image (Fig. 5)')
axes[0, 0].axis('off')

axes[0, 1].imshow(img_equalized, cmap='gray', vmin=0, vmax=255)
axes[0, 1].set_title('Equalized Image')
axes[0, 1].axis('off')

axes[1, 0].hist(img.ravel(), 256, [0, 256], color='gray', alpha=0.8)
axes[1, 0].set_title('Original Histogram (Low Contrast)')
axes[1, 0].set_xlabel('Pixel Intensity')
axes[1, 0].set_ylabel('Pixel Count')
axes[1, 0].set_xlim([0, 255])
axes[1, 0].grid(True, linestyle='--', alpha=0.5)

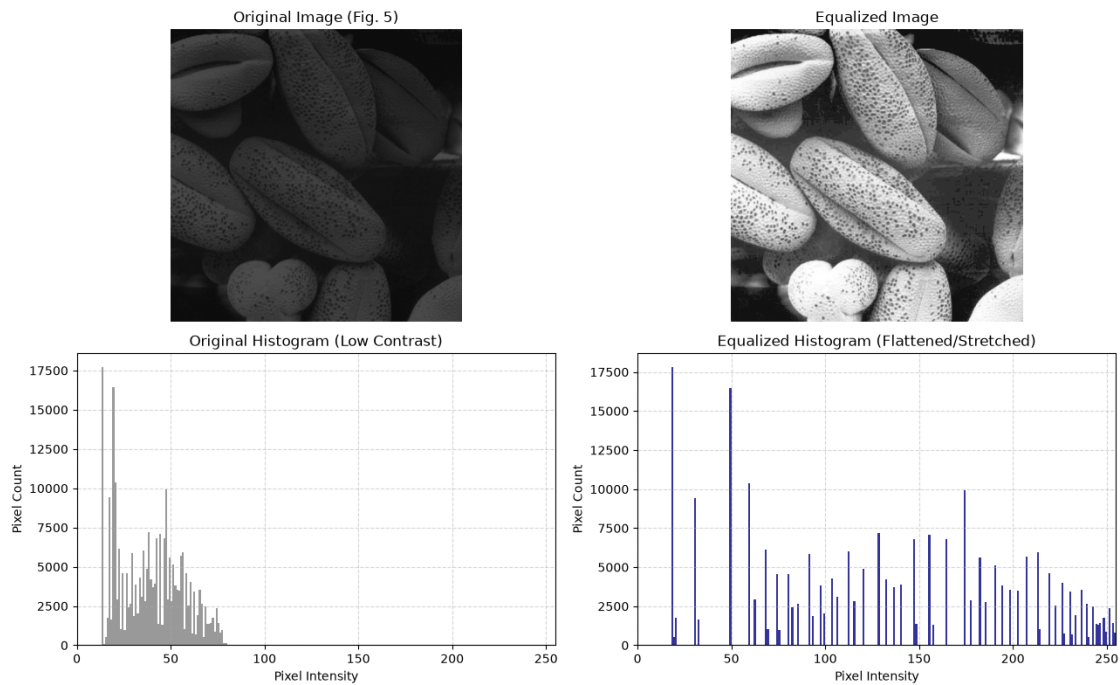
axes[1, 1].hist(img_equalized.ravel(), 256, [0, 256], color='navy', alpha=0.8)
axes[1, 1].set_title('Equalized Histogram (Flattened/Stretched)')
axes[1, 1].set_xlabel('Pixel Intensity')
```

```

axes[1, 1].set_ylabel('Pixel Count')
axes[1, 1].set_xlim([0, 255])
axes[1, 1].grid(True, linestyle='--', alpha=0.5)

plt.tight_layout()
plt.show()

```



Observation: Histogram equalization improves the overall contrast and makes more image details visible.

0.7 Q 06

```

[8]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img = cv.imread('images/q6.jpeg')
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
h, s, v = cv.split(hsv)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(h, cmap='gray'); axes[0].set_title('Hue Plane'); axes[0].
    axis('off')
axes[1].imshow(s, cmap='gray'); axes[1].set_title('Saturation Plane'); axes[1].
    axis('off')

```

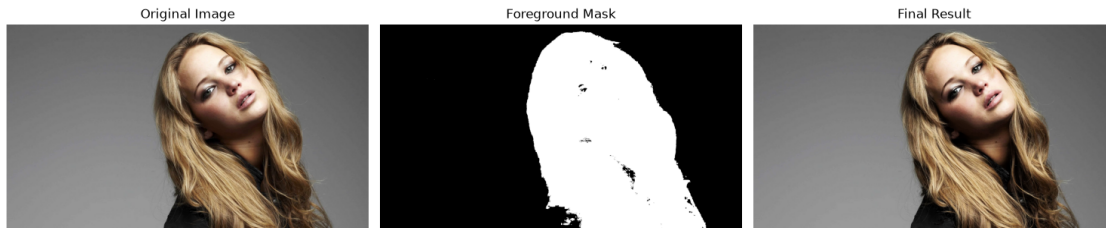


```
axes[2].imshow(v, cmap='gray');axes[2].set_title('Value Plane');axes[2].
    axis('off')
plt.tight_layout();plt.show()
```



```
[9]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

_, mask = cv.threshold(s, 15, 255, cv.THRESH_BINARY)
mask_inv = cv.bitwise_not(mask)
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
extracted_fg_bgr = cv.bitwise_and(img, img, mask=mask)
extracted_fg_rgb = cv.cvtColor(extracted_fg_bgr, cv.COLOR_BGR2RGB)
fg_v_pixels = v[mask > 0]
hist, _ = np.histogram(fg_v_pixels, bins=256, range=[0, 256])
cum_hist = np.cumsum(hist)
L = 256
MN = np.sum(mask > 0)
lut = np.round(((L - 1) / MN) * cum_hist).astype(np.uint8)
v_eq = lut[v]
v_eq_fg = cv.bitwise_and(v_eq, v_eq, mask=mask)
bg_bgr = cv.bitwise_and(img, img, mask=mask_inv)
bg_rgb = cv.cvtColor(bg_bgr, cv.COLOR_BGR2RGB)
hsv_eq = cv.merge([h, s, v_eq])
eq_fg_bgr = cv.bitwise_and(cv.cvtColor(hsv_eq, cv.COLOR_HSV2BGR), cv.
    cv.cvtColor(hsv_eq, cv.COLOR_HSV2BGR), mask=mask)
eq_fg_rgb = cv.cvtColor(eq_fg_bgr, cv.COLOR_BGR2RGB)
final_bgr = cv.add(eq_fg_bgr, bg_bgr)
final_rgb = cv.cvtColor(final_bgr, cv.COLOR_BGR2RGB)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(img_rgb);axes[0].set_title('Original Image');axes[0].axis('off')
axes[1].imshow(mask, cmap='gray');axes[1].set_title('Foreground Mask');axes[1].
    axis('off')
axes[2].imshow(final_rgb);axes[2].set_title('Final Result');axes[2].axis('off')
plt.tight_layout();plt.show()
```



Observation: The Saturation plane clearly separates the colorful foreground from the less colorful background.

0.8 Q 07

```
[10]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img = cv.imread('images/q7.jpeg', cv.IMREAD_GRAYSCALE)
img_float = img.astype(np.float64)
Kernel_a = np.array([[ 1,  0, -1],[ 2,  0, -2],[ 1,  0, -1]], dtype=np.float64)
out_a = cv.filter2D(img_float, -1, Kernel_a)

def custom_filter2d(image: np.ndarray, kernel: np.ndarray) -> np.ndarray:
    h_img, w_img = image.shape
    h_k, w_k = kernel.shape
    pad_h, pad_w = h_k // 2, w_k // 2
    padded = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant')
    #zero padded
    output = np.zeros_like(image, dtype=np.float64)
    k_flip = np.flipud(np.fliplr(kernel))
    for i in range(h_img):
        for j in range(w_img):
            region = padded[i:i + h_k, j:j + w_k]
            output[i, j] = np.sum(region * k_flip)

    return output

out_b = custom_filter2d(img_float, Kernel_a)
v_kernel = np.array([[1], [2], [1]], dtype=np.float64) # 3x1 vertical
#vector
h_kernel = np.array([[1, 0, -1]], dtype=np.float64) # 1x3 horizontal
#vector
step1 = cv.filter2D(img_float, -1, h_kernel)
out_c = cv.filter2D(step1, -1, v_kernel)
fig, axes = plt.subplots(1, 4, figsize=(18, 5))
```



```

axes[0].imshow(img, cmap='gray');axes[0].set_title('Original Image');axes[0].
    ↪axis('off')
axes[1].imshow(np.abs(out_a), cmap='gray');axes[1].set_title('(a) cv.filter2D');
    ↪axes[1].axis('off')
axes[2].imshow(np.abs(out_b), cmap='gray');axes[2].set_title('(b) Custom 2D_
    ↪Sobel');axes[2].axis('off')
axes[3].imshow(np.abs(out_c), cmap='gray');axes[3].set_title('(c) Separable 1D_
    ↪Sobel');axes[3].axis('off')
plt.tight_layout();plt.show()
diff_ab = np.max(np.abs(out_a - out_b))
diff_ac = np.max(np.abs(out_a - out_c))
print(f"Max Absolute Difference (a vs b): {diff_ab:.6f}")
print(f"Max Absolute Difference (a vs c): {diff_ac:.6f}")

```



Max Absolute Difference (a vs b): 1484.000000

Max Absolute Difference (a vs c): 0.000000

0.9 Q 08

```

[11]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

def zoom_image(img: np.ndarray, s: float, method: str = 'bilinear') -> np.
    ↪ndarray:
    H_in, W_in = img.shape[:2]
    H_out, W_out = int(np.round(H_in * s)), int(np.round(W_in * s))
    r_out, c_out = np.meshgrid(np.arange(H_out), np.arange(W_out),
    ↪indexing='ij')
    r_in = r_out / s
    c_in = c_out / s

    if method == 'nearest':
        r_idx = np.clip(np.round(r_in).astype(int), 0, H_in - 1)
        c_idx = np.clip(np.round(c_in).astype(int), 0, W_in - 1)

```

```

        return img[r_idx, c_idx]

    elif method == 'bilinear':
        r0 = np.clip(np.floor(r_in).astype(int), 0, H_in - 1)
        c0 = np.clip(np.floor(c_in).astype(int), 0, W_in - 1)
        r1 = np.clip(r0 + 1, 0, H_in - 1)
        c1 = np.clip(c0 + 1, 0, W_in - 1)
        dr = r_in - r0
        dc = c_in - c0
        if img.ndim == 3:
            dr = dr[..., np.newaxis]
            dc = dc[..., np.newaxis]
        w00 = (1 - dr) * (1 - dc)
        w10 = dr * (1 - dc)
        w01 = (1 - dr) * dc
        w11 = dr * dc
        out = (w00 * img[r0, c0] + w10 * img[r1, c0] + w01 * img[r0, c1] + w11 *
        ↪ img[r1, c1])
        return np.clip(out, 0, 255).astype(np.uint8)
    else:
        raise ValueError("Method must be 'nearest' or 'bilinear'")

def compute_normalized_ssd(img_orig: np.ndarray, img_zoomed: np.ndarray) -> ↪
    ↪ float:
    h = min(img_orig.shape[0], img_zoomed.shape[0])
    w = min(img_orig.shape[1], img_zoomed.shape[1])
    i_orig = img_orig[:h, :w].astype(np.float64)
    i_zoom = img_zoomed[:h, :w].astype(np.float64)
    ssd = np.sum((i_orig - i_zoom) ** 2)
    norm = np.sum(i_orig ** 2)
    return float(ssd / norm)

```

```

[12]: image_pairs = [('images/im01.png', 'images/im01small.png', '1st image'),
                    ('images/im02.png', 'images/im02small.png', '2nd image'),
                    ('images/im03.png', 'images/im03small.png', '3rd image'),
                    ('images/im04.jpg', 'images/im04small.jpg', '4th image')]

scale_factor = 4.0
print(f"{'Image Pair':<20} | {'Nearest-Neighbor SSD':<22} | {'Bilinear SSD':
    ↪ <22}")
print("-" * 70)
fig, axes = plt.subplots(len(image_pairs), 4, figsize=(18, 4 *
    ↪ len(image_pairs)))
for idx, (orig_path, small_path, label) in enumerate(image_pairs):
    orig_bgr = cv.imread(orig_path)
    small_bgr = cv.imread(small_path)
    orig_rgb = cv.cvtColor(orig_bgr, cv.COLOR_BGR2RGB)
    small_rgb = cv.cvtColor(small_bgr, cv.COLOR_BGR2RGB)

```

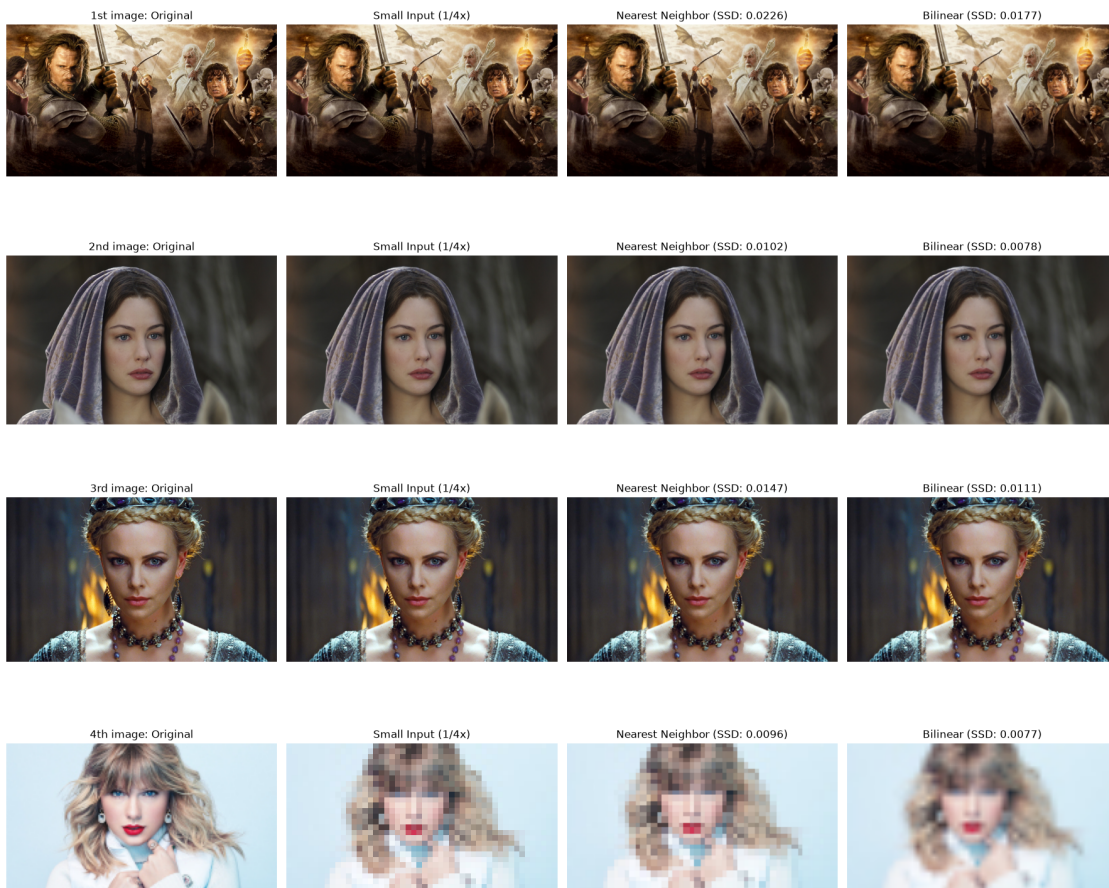
```

zoomed_nn = zoom_image(small_rgb, s=scale_factor, method='nearest')
zoomed_bl = zoom_image(small_rgb, s=scale_factor, method='bilinear')
ssd_nn = compute_normalized_ssd(orig_rgb, zoomed_nn)
ssd_bl = compute_normalized_ssd(orig_rgb, zoomed_bl)
print(f"{label:<20} | {ssd_nn:<22.6f} | {ssd_bl:<22.6f}")
axes[idx, 0].imshow(orig_rgb);axes[idx, 0].set_title(f'{label}: Original');
↪axes[idx, 0].axis('off')
axes[idx, 1].imshow(small_rgb);axes[idx, 1].set_title('Small Input (1/4x)');
↪axes[idx, 1].axis('off')
axes[idx, 2].imshow(zoomed_nn);axes[idx, 2].set_title(f'Nearest Neighbor_
↪(SSD: {ssd_nn:.4f})');axes[idx, 2].axis('off')
axes[idx, 3].imshow(zoomed_bl);axes[idx, 3].set_title(f'Bilinear (SSD:
↪{ssd_bl:.4f})');axes[idx, 3].axis('off')

plt.tight_layout();plt.show()

```

Image Pair	Nearest-Neighbor SSD	Bilinear SSD
1st image	0.022590	0.017719
2nd image	0.010240	0.007757
3rd image	0.014717	0.011126
4th image	0.009599	0.007711



Observation: Bilinear interpolation produces smoother results and lower error than nearest-neighbor interpolation.

0.10 Q 09

```
[13]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img_bgr = cv.imread('images/q9.jpeg')
img_rgb = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)
h, w, _ = img_bgr.shape
mask = np.zeros((h, w), np.uint8)
bgdModel = np.zeros((1, 65), np.float64)
fgdModel = np.zeros((1, 65), np.float64)
rect = (int(w * 0.05), int(h * 0.1), int(w * 0.95), int(h * 0.85))
cv.grabCut(img_bgr, mask, rect, bgdModel, fgdModel, 5, cv.GC_INIT_WITH_RECT)
mask_binary = np.where((mask == 1) | (mask == 3), 1, 0).astype('uint8')
fg_img = img_rgb * mask_binary[:, :, np.newaxis]
bg_img = img_rgb * (1 - mask_binary[:, :, np.newaxis])

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(mask_binary, cmap='gray'); axes[0].set_title('(a) Final_
↳Segmentation Mask'); axes[0].axis('off')
axes[1].imshow(fg_img); axes[1].set_title('(a) Foreground Image'); axes[1].
↳axis('off')
axes[2].imshow(bg_img); axes[2].set_title('(a) Background Image'); axes[2].
↳axis('off')
plt.tight_layout(); plt.show()

blurred_bg = cv.GaussianBlur(img_rgb, (51, 51), 0)
enhanced_img = np.where(mask_binary[:, :, np.newaxis] == 1, img_rgb, blurred_bg)
fig, axes = plt.subplots(1, 2, figsize=(8, 4))
axes[0].imshow(img_rgb); axes[0].set_title('Original Image'); axes[0].axis('off')
axes[1].imshow(enhanced_img); axes[1].set_title('(b) Enhanced (Substantially_
↳Blurred Background)'); axes[1].axis('off')
plt.tight_layout(); plt.show()
```



0.10.1 Explanation

- **Boundary Zero Value Bleeding:** Since the foreground color is black, there will be black color bleeding when the image is blurred. The result is a dark edge around the flower.
- **Shadow Spread Effect:** Shadows on the background behind the flower have been spread around the flower boundary due to Gaussian blur.

0.11 Q 10

```
[14]: import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt

img = cv.imread('images/q10.jpeg')
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
d = 9 # Diameter of pixel neighborhood
sigma_r = 75 # Filter sigma in color space
sigma_s = 75 # Filter sigma in coordinate space
gaussian_blurred = cv.GaussianBlur(img_rgb, (d, d), sigmaX=sigma_s,
    ↪sigmaY=sigma_s)
opencv_bilateral = cv.bilateralFilter(img_rgb, d=d, sigmaColor=sigma_r,
    ↪sigmaSpace=sigma_s)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(img_rgb); axes[0].set_title('Original Image'); axes[0].axis('off')
axes[1].imshow(gaussian_blurred); axes[1].set_title(f'Gaussian Blur_
    ↪(kernel={d}x{d}, sigma={sigma_s})'); axes[1].axis('off')
axes[2].imshow(opencv_bilateral); axes[2].set_title(f'OpenCV Bilateral (d={d},
    ↪sigma_r={sigma_r}, sigma_s={sigma_s})'); axes[2].axis('off')
plt.tight_layout(); plt.savefig('q10_part_a.png'); plt.show()

def custom_bilateral_filter(image, d, sigma_color, sigma_space):
    radius = d // 2
    pad_img = np.pad(image.astype(np.float64), ((radius, radius), (radius,
    ↪radius), (0, 0)), mode='reflect')
    out_img = np.zeros_like(image, dtype=np.float64)
    y, x = np.mgrid[-radius:radius+1, -radius:radius+1]
    spatial_weights = np.exp(-(x**2 + y**2) / (2.0 * sigma_space**2))
    height, width, channels = image.shape
    for i in range(height):
        for j in range(width):
            region = pad_img[i:i + d, j:j + d, :]
            center_pixel = pad_img[i + radius, j + radius, :]
            color_diff = region - center_pixel
            range_weights = np.exp(-np.sum(color_diff**2, axis=2) / (2.0 *
    ↪sigma_color**2))
            combined_weights = spatial_weights * range_weights
            sum_weights = np.sum(combined_weights)
            for c in range(channels):
                out_img[i, j, c] = np.sum(region[:, :, c] * combined_weights) /
    ↪sum_weights

    return np.clip(out_img, 0, 255).astype(np.uint8)
```



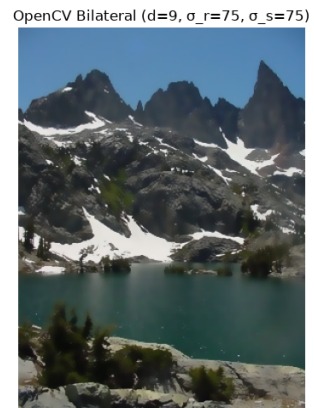
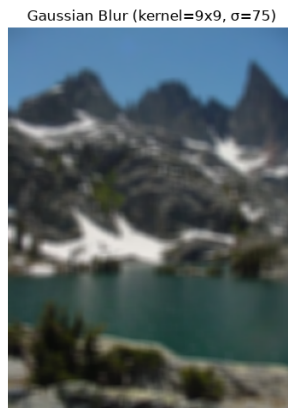
```

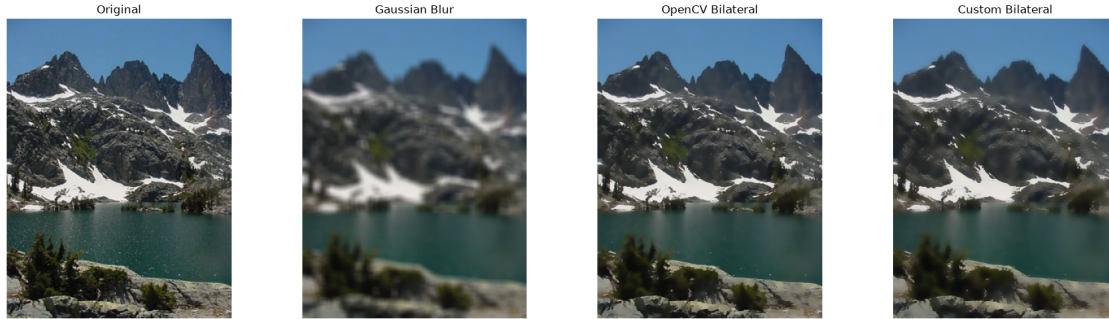
custom_bilateral = custom_bilateral_filter(img_rgb, d=d, sigma_color=sigma_r,
↪sigma_space=sigma_s)
fig, axes = plt.subplots(1, 4, figsize=(18, 5))
axes[0].imshow(img_rgb);axes[0].set_title('Original');axes[0].axis('off')
axes[1].imshow(gaussian_blurred);axes[1].set_title('Gaussian Blur');axes[1].
↪axis('off')
axes[2].imshow(opencv_bilateral);axes[2].set_title('OpenCV Bilateral');axes[2].
↪axis('off')
axes[3].imshow(custom_bilateral);axes[3].set_title('Custom Bilateral');axes[3].
↪axis('off')
plt.tight_layout();plt.savefig('q10_part_b.png');plt.show()

def calculate_psnr(img1, img2):
    mse = np.mean((img1.astype(np.float64) - img2.astype(np.float64)) ** 2)
    if mse == 0:
        return float('inf')
    max_pixel = 255.0
    return 20 * np.log10(max_pixel / np.sqrt(mse))

mse_val = np.mean((opencv_bilateral.astype(np.float64) - custom_bilateral.
↪astype(np.float64)) ** 2)
psnr_val = calculate_psnr(opencv_bilateral, custom_bilateral)
print("=== Quantitative Comparison (OpenCV vs Custom) ===")
print(f"Mean Squared Error (MSE): {mse_val:.4f}")
print(f"Peak Signal-to-Noise Ratio (PSNR): {psnr_val:.2f} dB")

```





=== Quantitative Comparison (OpenCV vs Custom) ===
Mean Squared Error (MSE): 29.0938
Peak Signal-to-Noise Ratio (PSNR): 33.49 dB

0.11.1 Discussion

The Gaussian blur blurs the entire image, including critical edges of the mountains and lake.

The bilateral filter helps to reduce the noise while keeping the edges preserved. Hence, the mountains and other objects' edges appear clearer than Gaussian filter.

0.11.2 Quantitative Evaluation

The performance of custom bilateral filter was compared with the OpenCV `bilateralFilter()`.

- **MSE:** 29.0938
- **PSNR:** 33.49 dB

The low MSE and high PSNR value indicate that the custom filter provides output close to the OpenCV's bilateral filter.

Conclusion: Bilateral filter performs edge preservation better than the Gaussian filter, and the custom filter result is close to the OpenCV bilateral filter result.